TikuKits ML: Tiny Neural Network for Embedded Systems

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

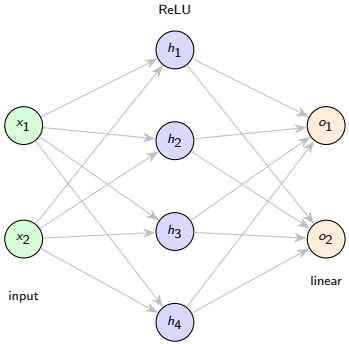
March 14, 2026

Outline

Why Neural Networks on MCUs?

Embedded Use Cases

- **Non-linear boundaries** — what linear models cannot solve
- **Multi-class** — temperature zones, gesture types
- **On-device training** — adapt to local conditions
- **Universal approximation** — learn any mapping



Constraints

- No FPU (16-bit MSP430)
- No heap allocation
- 320-byte stack on MSP430

Design Goal

Single-hidden-layer MLP with **ReLU activation**, **backpropagation**, and all computation in integer fixed-point arithmetic

Feedforward Neural Network

Forward pass:

Hidden layer (ReLU activation):

$$h_j = \text{ReLU}\left(w^h_{j,0} + \sum_{i=1}^{n} w^h_{j,i} \cdot x_i \right)$$

where $\text{ReLU}(z) = \max(0, z)$.

Output layer (linear):

$$o_k = w^o_{k,0} + \sum_{j=1}^{m} \frac{w^o_{k,j} \cdot h_j}{2^{\text{shift}}}$$

Prediction: $\hat{y} = \arg\max_k \; o_k$

Target: One-hot encoding
$t_k = 2^{\text{shift}}$ if $k = y$, else 0.

Backpropagation (SGD):

Output error:

$$\delta^o_k = o_k - t_k$$

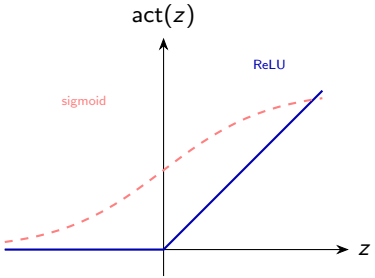
Hidden error (uses **old** output weights):

$$\delta^h_j = \begin{cases} \sum_k \frac{w^o_{k,j+1} \cdot \delta^o_k}{2^{\text{shift}}} & \text{if } h_j > 0 \\ 0 & \text{otherwise} \end{cases}$$

Critical: Update Order

Hidden weights must be updated **before** output weights — otherwise, error propagation uses modified weights and learning fails.

Why ReLU for Embedded?



ReLU Advantages for MCU

1. **No transcendentals:**
   $\mathrm{ReLU}(z) = \max(0, z)$
   One comparison, no `exp()`.
2. **Integer-friendly derivative:**
   $$\mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$
3. **No saturation:**
   Positive values pass through unchanged.
4. **Sparse activation:**
   Negative inputs produce zero — less work.

The tiku_kits_ml_tnn Structure

ml/classification/tiku_kits_ml_tnn.h

```c
struct tiku_kits_ml_tnn {
    /* Hidden weights: [hidden][input+1] */
    int32_t w_hidden
        [TIKU_KITS_ML_TNN_MAX_HIDDEN]
        [TIKU_KITS_ML_TNN_MAX_INPUT + 1];
    /* Output weights: [output][hidden+1] */
    int32_t w_output
        [TIKU_KITS_ML_TNN_MAX_OUTPUT]
        [TIKU_KITS_ML_TNN_MAX_HIDDEN + 1];
    /* Scratch: activations */
    int32_t h[TIKU_KITS_ML_TNN_MAX_HIDDEN];
    int32_t o[TIKU_KITS_ML_TNN_MAX_OUTPUT];

    uint8_t  n_input, n_hidden, n_output;
    uint8_t  shift;
    int32_t  learning_rate;
    uint16_t n;
};
```

Configuration macros

```c
#ifndef TIKU_KITS_ML_TNN_MAX_INPUT
#define TIKU_KITS_ML_TNN_MAX_INPUT  8
#endif
#ifndef TIKU_KITS_ML_TNN_MAX_HIDDEN
#define TIKU_KITS_ML_TNN_MAX_HIDDEN 8
#endif
#ifndef TIKU_KITS_ML_TNN_MAX_OUTPUT
#define TIKU_KITS_ML_TNN_MAX_OUTPUT 4
#endif
```

Memory footprint:

$8 \times 9 \times 4 + 4 \times 9 \times 4 + 8 \times 4 + 4 \times 4 + 10$
$= 442$ bytes (default config).

Stack Safety

Scratch buffers `h[]` and `o[]` are stored **in the struct**, not on the stack — critical for MSP430's 320-byte stack limit.

Complete API

Function	Description
`tiku_kits_ml_tnn_init(tnn, ni, nh, no, sh)`	Initialize with `ni` inputs, `nh` hidden, `no` outputs
`tiku_kits_ml_tnn_reset(tnn)`	Reinitialize weights, reset training count
`tiku_kits_ml_tnn_set_lr(tnn, rate)`	Set learning rate in Q(shift)
`tiku_kits_ml_tnn_train(tnn, x, y)`	One backpropagation SGD step; label $y \in [0, \text{n_output})$
`tiku_kits_ml_tnn_predict(tnn, x, &result)`	Forward pass + argmax: returns predicted class
`tiku_kits_ml_tnn_forward(tnn, x, scores)`	Forward pass only: returns all output scores
`tiku_kits_ml_tnn_get_weights(tnn, layer, w)`	Export weights (layer 0=hidden, 1=output)
`tiku_kits_ml_tnn_set_weights(tnn, layer, w)`	Load pre-trained weights
`tiku_kits_ml_tnn_count(tnn)`	Number of training samples seen

Minimum Classes

`init()` requires n_output ≥ 2 (classification needs at least 2 classes). Default learning rate: $(1 \ll \text{shift})/20 \approx 0.05$ in Q8 $= 12$.

Weight Initialization

Symmetry-breaking initialization

```
static void init_weights(
    struct tiku_kits_ml_tnn *tnn)
{
    uint8_t j, i, k;
    int32_t scale;

    /* ~1/n_input in Q(shift) */
    scale = (int32_t)(
        (1 << tnn->shift)
        / (tnn->n_input + 1));
    if (scale < 1) scale = 1;

    /* Hidden: alternating +/- scale */
    for (j = 0; j < tnn->n_hidden; j++) {
        tnn->w_hidden[j][0] = 0; /* bias */
        for (i = 1; i <= tnn->n_input; i++){
            tnn->w_hidden[j][i] =
                ((j + i) & 1)
                    ? scale : -scale;
        }
    }

    /* Output: all zero */
    for (k = 0; k < tnn->n_output; k++)
```

Why Alternating Signs?

- If all hidden weights are identical, all neurons compute the same function — **symmetry problem**
- Alternating $\pm$ ensures different neurons respond to different aspects of the input
- Scale $\approx 1/n$ prevents initial saturation
- No randomness needed (deterministic, reproducible)

Output Layer: All Zeros

Output weights start at zero so all classes begin with equal score. The network learns to differentiate through backpropagation.

Forward Pass and Backpropagation

Forward pass (hidden + output)

```
/* Hidden layer: ReLU */
for (j = 0; j < tnn->n_hidden; j++) {
    acc = (int64_t)tnn->w_hidden[j][0];
    for (i = 0; i < tnn->n_input; i++) {
        acc += (int64_t)tnn->w_hidden[j][i+1]
                * (int64_t)x[i];
    }
    tnn->h[j] = (acc > 0)
        ? (int32_t)acc : 0;
}

/* Output layer: linear */
for (k = 0; k < tnn->n_output; k++) {
    acc = (int64_t)tnn->w_output[k][0];
    for (j = 0; j < tnn->n_hidden; j++) {
        acc += ((int64_t)tnn->w_output[k][j+1]
                * (int64_t)tnn->h[j])
               >> tnn->shift;
    }
    tnn->o[k] = (int32_t)acc;
}
```

Backprop: hidden FIRST, then output

```
/* 1. Hidden gradients (old output wts) */
for (j = 0; j < tnn->n_hidden; j++) {
    if (tnn->h[j] <= 0) continue;
    err_sum = 0;
    for (k = 0; k < tnn->n_output; k++) {
        delta_o = tnn->o[k]
            - ((k == y) ? one_q : 0);
        err_sum += ((int64_t)
            tnn->w_output[k][j + 1]
            * (int64_t)delta_o)
            >> tnn->shift;
    }
    /* Update hidden weights */
    lr_delta = (lr_q * err_sum)
                >> tnn->shift;
    tnn->w_hidden[j][0] -= lr_delta;
    for (i = 0; i < tnn->n_input; i++)
        tnn->w_hidden[j][i+1] -=
            lr_delta * (int64_t)x[i];
}

/* 2. Output weights (safe to update) */
for (k = 0; k < tnn->n_output; k++) {
    delta_o = tnn->o[k]
```

Fixed-Point Arithmetic in the Network

Q-Format Flow

- **Inputs:** Integer scale (raw sensor values)
- **Hidden weights:** Q(shift)
- **Hidden activations:** $w \times x = \text{Q(shift)}$
- **Output weights:** Q(shift)
- **Output products:** $w^o \times h = \text{Q}(2\times\text{shift})$
  $\rightarrow$ right-shift by `shift` to get Q(shift)
- **Targets:** $1 \ll \text{shift}$ (one-hot in Q(shift))

Overflow Prevention

- All products computed as `int64_t`
- int32 $\times$ int32 $=$ int64
- Two-step `lr_delta` avoids triple products:
  1. $\text{lr_delta} = (\eta \times \delta) \gg \text{shift}$
  2. $w \mathrel{-}= \text{lr_delta} \times x$
- Safe for 8 inputs, 8 hidden, 4 outputs

Default Learning Rate

$(1 \ll 8)/20 = 12 \approx 0.047$ in Q8.
Typical range: 6–25 (0.025–0.1).

Example 1: Temperature Zone Classification

examples/example_ml_tnn.c — 1 input, 3 output classes

```c
struct tiku_kits_ml_tnn tnn;
uint8_t result;

/* 1 input, 4 hidden, 3 outputs, Q8 */
tiku_kits_ml_tnn_init(&tnn, 1, 4, 3, 8);
tiku_kits_ml_tnn_set_lr(&tnn, 6);   /* ~0.025 */

/* Training data: cold=0, normal=1, hot=2 */
static const int32_t temps[] = {50, 100, 150, 200, 250, 300, 350, 400, 450};
static const uint8_t labels[] = { 0,   0,   0,   1,   1,   1,   2,   2,   2};

uint16_t epoch;
uint8_t s;
for (epoch = 0; epoch < 80; epoch++) {
    for (s = 0; s < 9; s++) {
        tiku_kits_ml_tnn_train(&tnn, &temps[s], labels[s]);
    }
}

/* Predict */
int32_t query = 120;
tiku_kits_ml_tnn_predict(&tnn, &query, &result);  /* result = 0 (cold) */

query = 260;
```

Example 2: Gesture Recognition

2-D accel: push vs. pull

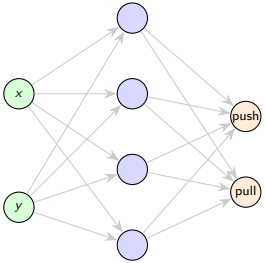
```
struct tiku_kits_ml_tnn tnn;
uint8_t result;
int32_t scores[2];

/* 2 inputs, 4 hidden, 2 outputs, Q8 */
tiku_kits_ml_tnn_init(&tnn, 2, 4, 2, 8);
tiku_kits_ml_tnn_set_lr(&tnn, 13);

/* Push (class 0): +X, small Y */
tiku_kits_ml_elem_t push1[] = {10, 2};
tiku_kits_ml_elem_t push2[] = {12, -1};

/* Pull (class 1): -X, small Y */
tiku_kits_ml_elem_t pull1[] = {-10, 1};
tiku_kits_ml_elem_t pull2[] = {-8, -2};

/* Train 50 epochs */
uint16_t ep;
for (ep = 0; ep < 50; ep++) {
    tiku_kits_ml_tnn_train(&tnn, push1, 0);
    tiku_kits_ml_tnn_train(&tnn, push2, 0);
    tiku_kits_ml_tnn_train(&tnn, pull1, 1);
    tiku_kits_ml_tnn_train(&tnn, pull2, 1);
}
```



ReLU

Raw Scores via `forward()`

`tnn_forward()` returns per-class output scores in Q(shift). Higher score = more likely. Use for:

- Confidence thresholding

Example 3: Pre-Trained Deployment

Load weights trained offline

```
struct tiku_kits_ml_tnn tnn;
uint8_t result;

tiku_kits_ml_tnn_init(&tnn, 1, 2, 2, 8);

/* Hidden layer: 2 neurons x (1 bias + 1 input) = 4 weights */
int32_t w_hidden[] = {
    0,  256,     /* neuron 0: ReLU(x)   */
    0, -256,     /* neuron 1: ReLU(-x)  */
};
tiku_kits_ml_tnn_set_weights(&tnn, 0, w_hidden);

/* Output layer: 2 classes x (1 bias + 2 hidden) = 6 weights */
int32_t w_output[] = {
    0,  256,    0,     /* class 0: high when ReLU(x) > 0  (x > 0) */
    0,    0,  256,     /* class 1: high when ReLU(-x) > 0 (x < 0) */
};
tiku_kits_ml_tnn_set_weights(&tnn, 1, w_output);

/* Predict */
int32_t pos = 5;
tiku_kits_ml_tnn_predict(&tnn, &pos, &result);    /* result = 0 */

int32_t neg = -5;
```

Test Cases Overview

#	Test Function	What It Verifies
1	`test_kits_ml_tnn_init`	Initialization, parameter bounds, n_output ≥ 2
2	`test_kits_ml_tnn_pretrained`	Hand-crafted weights: ReLU(x) vs ReLU(-x)
3	`test_kits_ml_tnn_forward_pass`	Raw output scores match expected values
4	`test_kits_ml_tnn_training`	2-input, 4-hidden, 2-output convergence (50 epochs)
5	`test_kits_ml_tnn_three_class`	1-input, 4-hidden, 3-output convergence (80 epochs)
6	`test_kits_ml_tnn_learning_rate`	Custom learning rate accepted
7	`test_kits_ml_tnn_weight_access`	Get/set weights round-trip (both layers)
8	`test_kits_ml_tnn_reset`	Reset reinitializes weights and count
9	`test_kits_ml_tnn_null_inputs`	All functions reject NULL with ERR_NULL

Summary

What We Built

- Single-hidden-layer feedforward network
- ReLU activation (integer-friendly)
- SGD backpropagation on-device
- ∼442 bytes state (default config)
- Multi-class via argmax of outputs

Key Design Choices

- Scratch buffers in struct (stack safety)
- Alternating weight init (symmetry breaking)
- Hidden-first backprop update order
- Two-step lr_delta prevents overflow

Use Cases

- Temperature zone classification (3+ classes)
- Gesture recognition (push/pull/tap)
- Non-linear decision boundaries
- Adaptive on-device learning

Files

Header:	`ml/classification/tiku_kits_ml_tnn.h`
Source:	`ml/classification/tiku_kits_ml_tnn.c`
Tests:	`tests/ml/test_kits_ml_tnn.c`
Example:	`examples/example_ml_tnn.c`

Thank You

TikuKits ML: Tiny Neural Network

Integer-Only Machine Learning for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
Common:	`tikukits/ml/tiku_kits_ml.h`
Header:	`tikukits/ml/classification/tiku_kits_ml_tnn.h`
Source:	`tikukits/ml/classification/tiku_kits_ml_tnn.c`
Tests:	`tikukits/tests/ml/test_kits_ml_tnn.c`
Examples:	`tikukits/examples/example_ml_tnn.c`